Travail d'étude et de recherche

Apprentissage d'une stratègie d'attaque contre un véhicule autonome

Réalisé par :

Rabah OUAFI (20223502@etud.univ-evry.fr)
Massilia BEGHDAD (20215721@etud.univ-evry.fr)
Saliha TABANI (20215023@etud.univ-evry.fr)

Encadré par :

Guillaume HUTZLER

En collaboration avec :

Witold KLAUDEL
Artur RATAJ
Hanna KLAUDEL
Franck POMMERREAU

15 mai 2025

Table des matières

1 Introduction **1**
1.1 Contexte du projet . . . 1
1.2 Problématique posée . . . 1
1.3 Objectif du travail d'étude et de recherche . . . 1
1.4 État de l'art . . . 2

2 Approches d'Apprentissage Automatique **2**
2.1 Définition du Q-Learning . . . 2
2.1.1 Fonctionnement général . . . 2
2.2 Algorithme génétique . . . 3
2.2.1 Principe d'évolution . . . 3
2.2.2 Adaptation aux systèmes dynamiques . . . 3
2.3 Les systèmes de classifieurs (LCS) . . . 4
2.3.1 Définition et fonctionnement . . . 4
2.3.2 Les types de LCS Michigan vs Pittsburg . . . 4

3 Le simulateur **5**
3.1 Description du simulateur . . . 5
3.2 Les fonctions proposées par le simulateur . . . 6

4 XCS : Système de Classifieurs Étendu **7**
4.1 Définition du XCS . . . 7
4.2 Architecture général . . . 7
4.3 Justification du choix du XCS pour notre projet . . . 7

5 Implémentation **8**
5.1 Implémentation de la classe de l'algorithme . . . 8
5.1.1 Structure générale . . . 8
5.1.2 Encodage des états . . . 8
5.1.3 Récupération et vérification des actions . . . 8
5.1.4 Exécution et récompense . . . 8
5.1.5 Contrôle du déroulement . . . 8
5.2 Étapes d'intégration de XCS dans le simulateur . . . 9

6 Expérimentations sur différents modèles **9**
6.1 Méthodologie . . . 9
6.2 Exemple de règle apprise par XCS . . . 9
6.3 Résultats . . . 10
6.4 Analyse de la précision d'apprentissage . . . 10

7 Conclusion **11**
7.1 Apports et limitations de XCS dans la simulation . . . 11
7.2 Pistes d'amélioration futures . . . 12
7.3 Conclusion finale . . . 12

1 Introduction

1.1 Contexte du projet

L'évolution rapide des technologies avancées intégrées aux véhicules autonomes soulève de nouveaux défis en matière de sécurité informatique. Ces véhicules sont contrôlés par un ensemble complexe de modules interconnectés, intégrant des capteurs, des algorithmes décisionnels, des systèmes de communication, ainsi que des actionneurs. Leur bon fonctionnement repose sur l'intégrité et la fiabilité de ces composants, qui sont constamment exposés à des risques de cyberattaques. Une perturbation même mineure dans l'un de ces modules peut engendrer une cascade de défaillances critiques à l'échelle du système global. Dans le cadre d'un travail de recherche sur l'analyse de risque et la sécurité des systèmes intelligents, un formalisme a été développé pour modéliser les interactions entre ces modules, notamment la manière dont un état défaillant peut se propager à travers le réseau du véhicule. Afin de dépasser les limites des approches classiques comme le Q-learning – dont la complexité devient prohibitive sur des systèmes de grande taille – le projet se tourne vers une solution alternative :**les systèmes de classeurs (Learning Classifier Systems)**. Ces algorithmes, qui combinent apprentissage par renforcement* et optimisation évolutionnaire, permettent d'explorer efficacement un grand espace de stratégies possibles. L'objectif est d'apprendre automatiquement des stratégies d'attaque simulées sur le véhicule, pour en déduire les vulnérabilités les plus critiques à corriger.

1.2 Problématique posée

Les véhicules autonomes, de par leur architecture complexe et interconnectée, sont particulièrement vulnérables aux cyberattaques, ce qui soulève la nécessité de développer des outils capables de simuler des attaques et d'identifier les failles de sécurité. Cependant, le défi majeur réside dans l'exploration de l'espace des vulnérabilités et la modélisation efficace des différentes stratégies d'attaque. Les Systèmes de Classificateurs d'Apprentissage (LCS), qui combinent les avantages des algorithmes évolutionnaires et de l'apprentissage par renforcement, semblent une solution pertinente pour cette tâche. Toutefois, la difficulté réside dans le choix du modèle LCS le plus adapté, ainsi que dans l'adaptation de cet algorithme afin de traiter des espaces de solutions de grande taille tout en identifiant de manière précise les vulnérabilités exploitables d'un véhicule autonome.

1.3 Objectif du travail d'étude et de recherche

Ce projet a pour objectif d'évaluer la pertinence des Systèmes de Classificateurs d'Apprentissage (LCS) pour l'identification de stratégies d'attaque efficaces contre un véhicule autonome, dans le but de détecter ses vulnérabilités. Pour ce faire, il s'agira de réaliser une revue des principales variantes d'algorithmes LCS, de sélectionner celle qui est la mieux adaptée au contexte des véhicules autonomes, puis de rechercher une implémentation open source de cet algorithme. L'algorithme sera ensuite adapté au simulateur existant afin de l'appliquer à un cas d'étude concret. L'approche permettra ainsi de tester la capacité des LCS à explorer efficacement l'espace des attaques et à révéler des points faibles dans l'architecture du véhicule autonome.

1.4 État de l'art

La cyber-sécurité des véhicules autonomes a fait l'objet de nombreuses recherches, notamment sur la génération automatique de stratégies d'attaque. Des approches comme les arbres d'attaque et l'apprentissage automatique, notamment le Q-Learning, permettent de simuler des attaques sur des systèmes complexes. Cependant, ces méthodes sont parfois difficiles à interpréter et à adapter aux spécificités des véhicules autonomes.

Dans ce contexte, l'utilisation des systèmes de classificateurs d'apprentissage (LCS) pourrait offrir une alternative prometteuse pour la détection proactive d'attaques dans les véhicules autonomes. Les LCS sont capables d'apprendre à partir d'interactions avec l'environnement et d'adapter leurs stratégies en temps réel, ce qui les rend particulièrement adaptés pour simuler et détecter des attaques dans des systèmes dynamiques et complexes comme les véhicules autonomes.

2 Approches d'Apprentissage Automatique

2.1 Définition du Q-Learning

L'algorithme Q-learning est une méthode d'apprentissage par renforcement dans laquelle un agent apprend à prendre des décisions optimales en interagissant avec un environnement. Il fait partie des algorithmes de type model-free, ce qui signifie qu'il ne nécessite pas de connaissance préalable sur la dynamique de l'environnement.
Le "Q" dans Q-learning fait référence à la qualité de l'action choisie par le modèle, qui vise à améliorer continuellement la performance en fonction des informations disponibles.[7]

2.1.1 Fonctionnement général

Le fonctionnement du Q-learning repose sur l'apprentissage et l'amélioration progressive de la fonction de valeur $Q(s, a)$ au fur et à mesure que l'agent interagit avec l'environnement. L'agent choisit une action à chaque étape en suivant une politique qui favorise l'exploitation des actions les plus prometteuses tout en permettant une exploration occasionnelle pour éviter de rester bloqué dans des solutions sous-optimales. Après avoir effectué une action et reçu une récompense, l'agent met à jour la valeur $Q(s_t, a_t)$ en fonction de la récompense obtenue et de l'estimation des récompenses futures, en utilisant la valeur maximale des actions possibles dans le nouvel état. Cette mise à jour continue jusqu'à ce que la fonction Q converge vers les valeurs optimales, permettant à l'agent de prendre des décisions qui maximisent la récompense cumulée à long terme. La mise à jour de $Q(s, a)$ suit la formule suivante :

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$$

[3][4]

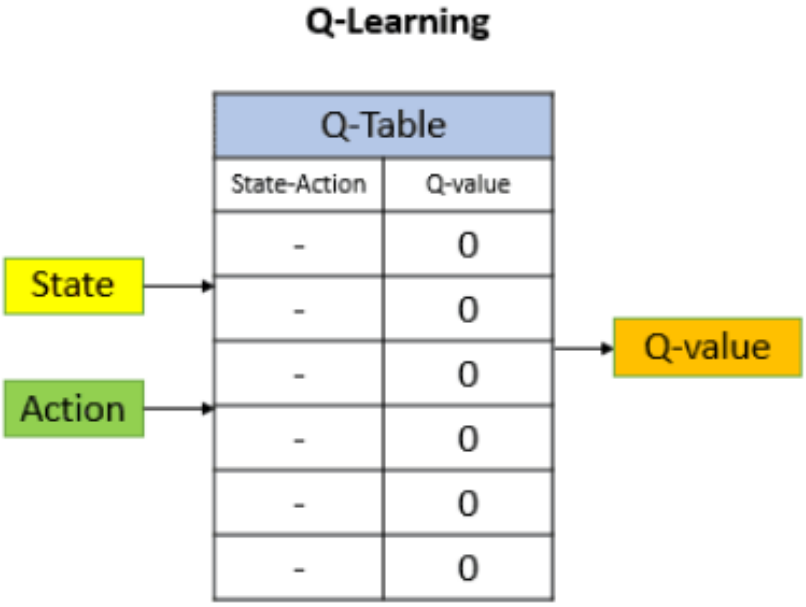

FIGURE 1 – Exemple d'une Q-table

2.2 Algorithme génétique

2.2.1 Principe d'évolution

L'algorithme génétique simule un processus de sélection naturelle à travers une population d'individus représentant des solutions possibles (génomes). Chaque génération passe par des étapes de sélection, croisement et mutation*, dans le but de faire émerger progressivement de meilleures solutions.

L'évolution se manifeste par l'amélioration continue de la population : après l'évaluation de chaque individu selon une fonction de fitness*, les plus adaptés sont sélectionnés pour engendrer une nouvelle génération. Les opérateurs de croisement permettent de combiner les caractéristiques de deux parents, tandis que la mutation introduit de la diversité, réduisant le risque de stagnation dans des optima locaux. Ce mécanisme évolutif favorise la préservation des bons traits tout en explorant de nouvelles combinaisons, assurant ainsi une recherche efficace de solutions optimales au fil du temps.

2.2.2 Adaptation aux systèmes dynamiques

L'adaptation de l'algorithme génétique aux systèmes dynamiques vise à permettre au système d'apprentissage de continuer à évoluer efficacement dans un environnement en constante évolution. Contrairement aux environnements statiques, où une solution optimale reste valable, les systèmes dynamiques exigent que l'algorithme soit capable de réagir à des changements imprévus ou progressifs. Dans ce contexte, plusieurs mécanismes sont utilisés : le maintien de la diversité génétique pour éviter la convergence prématurée, la détection des changements dans l'environnement, la réinitialisation partielle de la population pour réagir aux nouvelles conditions, et l'utilisation d'une mémoire évolutive permettant de réemployer des solutions efficaces dans des situations similaires. Ces adaptations permettent aux algorithmes génétiques, comme ceux intégrés dans les Sys-

tèmes de Classificateurs d'Apprentissage (LCS), de rester pertinents et performants dans des environnements complexes et instables tels que ceux des véhicules autonomes, où les vulnérabilités peuvent évoluer avec le temps et la structure du système.

2.3 Les systèmes de classifieurs (LCS)

2.3.1 Définition et fonctionnement

Les systèmes de classificateurs (LCS) sont des algorithmes d'apprentissage évolutifs qui combinent des techniques de l'apprentissage par renforcement et des algorithmes génétiques pour construire un ensemble de règles conditionnelles, appelées classificateurs. Chaque classificateur associe une condition à une action et possède une valeur de performance (fitness) indiquant son efficacité.

Un LCS fonctionne en plusieurs étapes. Tout d'abord, une population initiale de classificateurs est générée, souvent de manière aléatoire. À chaque interaction avec l'environnement, les classificateurs qui satisfont les conditions de leur règle sont activés et participent à la sélection de l'action à prendre. Après chaque action, un mécanisme de renforcement évalue la performance des classificateurs en fonction des récompenses ou des punitions reçues. Les classificateurs les plus performants ont une probabilité plus élevée d'être sélectionnés pour reproduire et générer de nouveaux classificateurs, en appliquant des opérateurs génétiques comme le croisement et la mutation. Ce processus évolutif permet au système de s'adapter aux changements dans l'environnement et d'améliorer la prise de décision au fil du temps.[5]

2.3.2 Les types de LCS Michigan vs Pittsburg

Dans l'approche Michigan, chaque individu de la population représente une seule règle (ou classificateur). L'ensemble des règles coopère pour résoudre le problème. L'évolution agit directement sur les règles, et le système apprend quelles règles doivent être renforcées ou éliminées. Cette approche est plus rapide et plus efficace pour les problèmes nécessitant des règles simples et modulaires, mais elle peut être sujette à des conflits ou à une mauvaise coordination entre règles.

À l'inverse, dans l'approche Pittsburgh, chaque individu représente un ensemble complet de règles, c'est-à-dire une solution autonome. L'évolution se fait sur des systèmes entiers, pas sur des règles individuelles. Cela permet une meilleure cohérence globale dans les décisions, au prix d'une plus grande complexité algorithmique, car chaque solution est plus lourde à évaluer.[1]

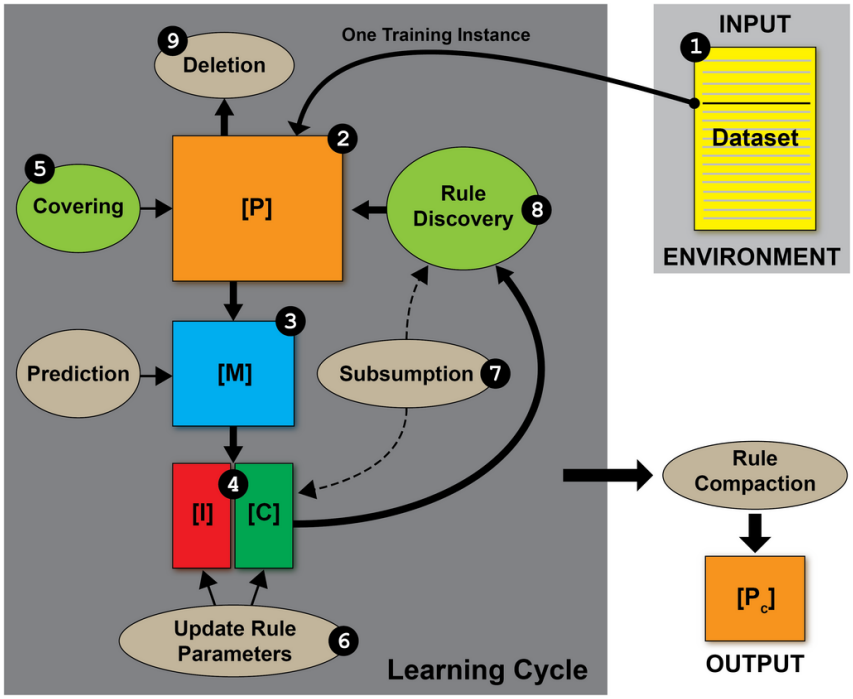

FIGURE 2 – Generic Michigan style Supervised LCS Schematic

3 Le simulateur

3.1 Description du simulateur

Le simulateur utilisé dans le cadre de ce projet permet de modéliser le comportement d'un véhicule autonome sous forme de graphe orienté où chaque nœud représente un module (ex. ADAS, ECU, capteurs), et les arêtes représentent les dépendances ou communications. Chaque module peut adopter différents états : FUNC (fonctionnel), MALW (infecté par un malware), BADD (corrompu), NOAV (éteint).Le simulateur lit une description du système à partir d'un fichier JSON et initialise l'environnement en fonction d'un état de départ spécifié manuellement ou via un fichier batch.

L'exécution se fait sous forme d'épisodes, où à chaque étape, le simulateur nous permet d'explorer le model via des transitions possibles selon un coût maximum prédéfini. Le but de chaque épisode est d'atteindre un état final cible tout en minimisant le coût total des transitions.

Le simulateur détecte également les situations de blocage (deadlocks) et identifie les séquences de transitions menant à l'objectif.[6]

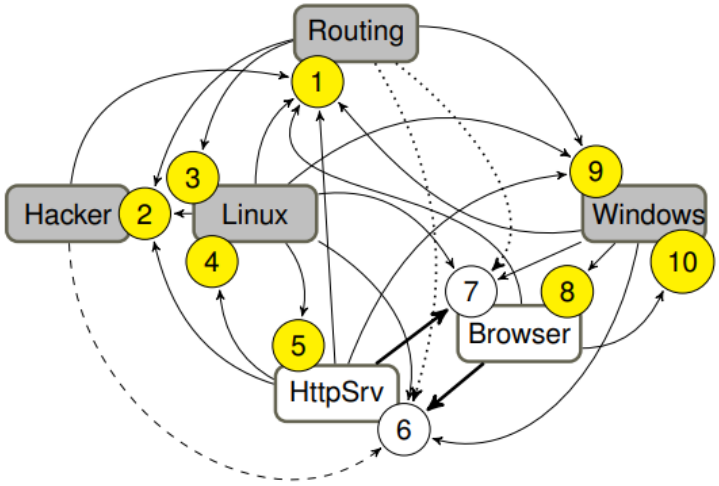

FIGURE 3 – The visibility graph of the toy example. Edges labelled mitm are dotted, side are dashed, and peer are solid thin. Functional edges are solid

3.2 Les fonctions proposées par le simulateur

Le simulateur nous met à disposition plusieurs fonctions et méthode afin de bien contrôler le model et la simulation, on retrouve donc :

— Fonction reset : afin de remettre le model à l'état initial à chaque épisode
— Fonction next(maxcost) : récupérer toutes les transitions possible à chaque pas selon le cout maximum
— Fonction Do : appliquer une transition choisie au système et appliquer les actions qu'elle fait
— Fonction CurrentLoc : permet de récuperer l'état de chaque module

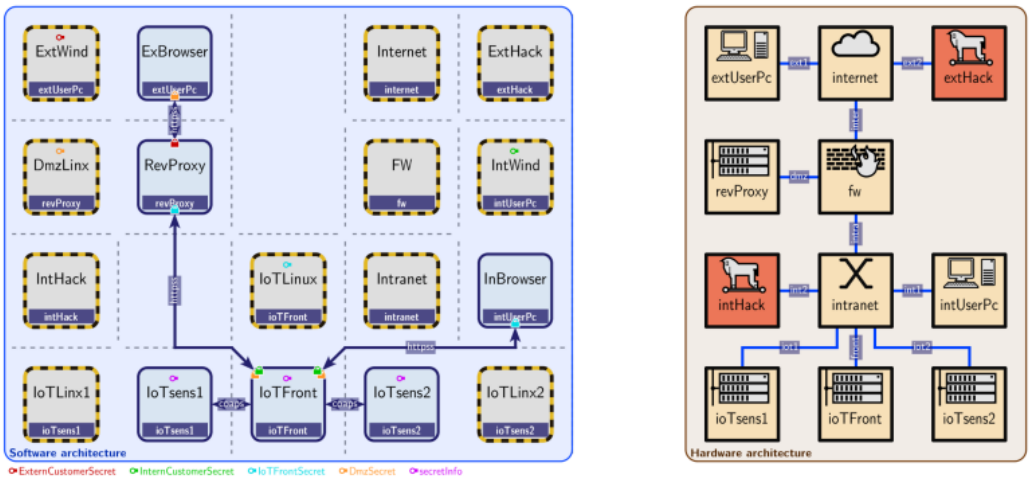

FIGURE 4 – Visualisation d'une architecture simplifié du simulateur

4 XCS : Système de Classifieurs Étendu

4.1 Définition du XCS

XCS (eXtended Classifier System) [2] est une extension avancée des systèmes de classificateurs d'apprentissage (LCS) qui intègre des principes plus rigoureux d'apprentissage par renforcement. Contrairement aux LCS classiques qui évaluent la performance des règles selon la récompense immédiate, XCS attribue la fitness des classificateurs en fonction de la précision de leurs prédictions. Cette approche encourage l'évolution de règles à la fois précises et généralisables, ce qui permet une meilleure adaptation à des environnements complexes et dynamiques. XCS se distingue également par une gestion plus fine de l'exploration/exploitation et une structure de mise à jour plus efficace.[8]

4.2 Architecture général

Le code XCS se structure autour de plusieurs composants clés :

- Classifier : chaque classifieur possède une condition (chaine binaire), une action (valeur entière représentant une décision), une prédiction (récompense estimée), une erreur, un fitness, et une expérience. Les classifieurs sont généralisés à travers un encodage symbolique ('dieze' pour indifférence).
- Population : XCS maintient un ensemble dynamique de classifieurs. Après chaque interaction avec l'environnement, de nouveaux classifieurs peuvent être insérés, croisés, mutés, ou supprimés selon leur utilité (fitness).
- Matching : à chaque pas de simulation, XCS sélectionne les classifieurs dont la condition correspond à l'état courant (via des correspondances de bits). Ce sous-ensemble constitue le match set.
- Prediction Array : chaque action possible est associée à une récompense attendue calculée via les classifieurs correspondants. L'action avec la meilleure prédiction est choisie selon une stratégie d'exploration/exploitation.
- Apprentissage : après exécution de l'action, une récompense est observée. Le action set (les règles responsables de cette action) est mis à jour : la prédiction, l'erreur et le fitness sont ajustés via des formules incrémentales.
- GA (Genetic Algorithm) : si les règles du action set ont suffisamment d'expérience, un algorithme génétique est appliqué pour croiser et muter deux règles, injectant ainsi de la diversité et favorisant la généralisation.

4.3 Justification du choix du XCS pour notre projet

Le choix de l'algorithme XCS dans le cadre de notre projet s'explique par ses caractéristiques particulièrement adaptées à l'apprentissage de stratégies d'attaque dans un système aussi complexe et dynamique qu'un véhicule autonome. XCS combine les avantages des algorithmes génétiques et de l'apprentissage par renforcement, ce qui en fait un candidat idéal pour explorer un espace de solutions très vaste comme celui des interactions entre les modules d'un véhicule. L'existence de bibliothèques open source en Python facilite son intégration avec le simulateur déjà disponible, ce qui permet de se concentrer sur l'adaptation fine de l'algorithme au problème. Il a également démontré son efficacité dans des environnements dynamiques et bruités, ce qui renforce sa pertinence pour identifier des vulnérabilités évolutives dans les systèmes de transport intelligents.[**?**]

5 Implémentation

5.1 Implémentation de la classe de l'algorithme

Pour interfacer l'algorithme XCS avec notre simulateur, nous avons défini une classe `CarHackScenario` dérivée de la classe abstraite* `Scenario` fournie par la librairie XCS. Cette classe agit comme une passerelle entre le monde symbolique des règles XCS (états encodés, identifiants d'action) et les mécanismes réels de simulation du modèle de véhicule.

5.1.1 Structure générale

Le constructeur de `CarHackScenario` initialise les éléments suivants :
- Le modèle d'automate* du véhicule (`automata`)
- Les états initiaux des modules (`init_loc`) et des secrets (`init_secr`)
- La cible de l'attaque (`target_index` et `target_state`)
- Les contraintes de simulation (`max_cost`, `max_step`)
- Les tables de correspondance entre transitions réelles et identifiants d'action (`action_encoder`, `action_decoder`)

5.1.2 Encodage des états

La méthode `sense()` encode l'état global du système sous forme de `BitString`. Chaque module est représenté par deux bits selon le mapping* suivant :

```
Location.FUNC -> [0, 0]
Location.MALW -> [0, 1]
Location.BADD -> [1, 0]
Location.NOAV -> [1, 1]
```

5.1.3 Récupération et vérification des actions

La méthode `get_possible_actions()` retourne tous les identifiants d'action connus. La faisabilité de l'action est vérifiée dynamiquement via `execute()`.

5.1.4 Exécution et récompense

La méthode `execute()` applique une transition valide et retourne une récompense basée sur :
- Le type d'action (ex. F2M, F2B, F2N, Fall)
- La compromission de secrets
- L'atteinte de la cible
- Le coût total de l'action

5.1.5 Contrôle du déroulement

La méthode `more()` détermine si l'épisode peut continuer. Elle s'arrête si :
- Le nombre maximal d'étapes est atteint
- Aucune transition n'est faisable
- La cible a été atteinte

5.2 Étapes d'intégration de XCS dans le simulateur

L'intégration de l'algorithme XCS s'est déroulée selon les étapes suivantes :

1. **Extraction des transitions du modèle** : toutes les transitions sont extraites de la structure JSON, sans filtrage.
2. **Encodage des états des modules** : chaque état de module est encodé comme suit :

```
Location.FUNC -> [0, 0]
Location.MALW -> [0, 1]
Location.BADD -> [1, 0]
Location.NOAV -> [1, 1]
```

3. **Récupération des actions possibles** : les transitions faisables à l'instant sont fournies à XCS sous forme d'identifiants.
4. **Sélection et vérification de l'action** : l'action sélectionnée par XCS est validée côté simulateur.
5. **Exécution et itération** : l'action est appliquée si faisable, et le cycle est relancé jusqu'à arrêt.

Ce processus permet à XCS d'apprendre progressivement quelles séquences d'actions sont optimales pour atteindre un module cible tout en naviguant dans un espace d'états complexe et dynamique.

6 Expérimentations sur différents modèles

6.1 Méthodologie

Pour évaluer les performances de l'algorithme XCS, nous avons testé notre implémentation sur les différents modèles de véhicules autonomes de tailles croissantes proposés par le simulateur :

— **VerySmall** : 6 modules et 30 transitions
— **Small** : 15 modules et 85 transitions
— **Medium** : 17 modules et 106 transitions
— **CarMonV5** : 78 modules et 395 transitions

Pour chaque modèle, 50 épisodes ont été lancés, avec les mêmes paramètres d'apprentissage (coût max, nombre de pas max, etc.). Le but est d'évaluer le taux de réussite (atteinte de la cible) ainsi que le temps moyen d'exécution.

6.2 Exemple de règle apprise par XCS

Voici un exemple concret de règle extraite après l'apprentissage sur le modèle `Medium` :

```
Condition        : 0000##00#0#00#001000#00####001#0##
Action ID        : 101
Prédiction       : 82.67
Erreur           : 29.48
Fitness          : 0.39
```

```
Expérience       : 6
Timestamp        : 22711
Action Set Size  : 2.67
Numerosity       : 1
```

Interprétation :

- **Condition** : cette chaîne binaire (avec caractères `#`) représente l'état global du véhicule. Chaque module est codé sur deux bits (00, 01, 10 ou 11 selon qu'il est `FUNC`, `MALW`, `BADD` ou `NOAV`). Le caractère `#` agit comme un joker (0 ou 1), permettant à la règle de s'appliquer dans plusieurs états similaires. Cela favorise la généralisation.
- **Action ID 101** : représente la transition numéro 101 codée dans le simulateur. Ce code numérique est interprété à l'aide du dictionnaire `action_decoder` qui répertorie les transitions selon leur ID.
- **Prédiction (82.67)** : c'est la récompense que la règle estime obtenir lorsqu'elle est utilisée. Ici, elle pense que l'action 101 dans l'état décrit lui rapportera environ 83 points.
- **Erreur (29.48)** : indique que la règle a encore une incertitude notable sur la précision de sa prédiction. Cela peut être dû à un faible nombre d'exécutions.
- **Fitness (0.39)** : relativement faible, ce qui signifie que cette règle est encore en phase d'exploration ou qu'elle manque de fiabilité par rapport aux autres.
- **Expérience (6)** : cette règle a été utilisée 6 fois. Ce n'est pas suffisant pour considérer ses statistiques comme totalement stabilisées.
- **Numerosity (1)** : une seule copie de cette règle existe dans la population. Cela confirme qu'elle n'a pas encore été beaucoup renforcée par l'algorithme génétique.

Conclusion : Cette règle semble prometteuse avec une prédiction élevée, mais manque encore de précision et d'expérience. Si son erreur diminue au fil du temps, elle pourrait devenir une règle dominante dans la stratégie d'attaque apprise par XCS.

6.3 Résultats

Les résultats sont synthétisés dans le tableau suivant :

Modèle	Taux de réussite (sur 50)	Temps moyen par épisode
VerySmall	96%	~ 0.2 secondes
Small	80%	~ 0.3 secondes
Medium	40%	~ 0.7 secondes
ResilientCar	0%	> 2 secondes

6.4 Analyse de la précision d'apprentissage

- Sur les modèles **VerySmall** et **Small**, XCS parvient à apprendre efficacement des stratégies atteignant la cible dans 96%, 80% des cas, avec un temps d'exécution très court.
- Sur le modèle **Medium**, le taux de succès chute à 20%. Cela s'explique par la complexité accrue de l'environnement : davantage de modules impliquent un espace d'état bien plus vaste, rendant l'apprentissage plus lent et moins fiable.

— Le modèle **ResilientCar** révèle les principales limites de XCS. Le temps d'exécution par épisode devient très long (plusieurs secondes), et aucun apprentissage significatif n'a pu être observé après 10000 itérations. La taille du modèle rend l'espace de recherche trop large pour être exploré efficacement sans mécanisme d'abstraction. Cependant, avec un nombre d'itération plus élevé et un paramétrage plus affiné on peut supposer que certains résultats peuvent apparaître.

Analyse des positions d'attaque privilégiées

En complément de l'étude des stratégies d'attaque, nous avons examiné les **positions d'entrée** des transitions choisies par l'algorithme XCS. Chaque règle apprise a été associée à la position du module cible de la transition correspondante (par exemple : `peer`, `side`, `mitm`).

En pondérant les occurrences par la valeur de `numerosity` des règles, nous obtenons un score global par position, reflétant leur importance dans les stratégies efficaces apprises par le système.

— `peer` : 105
— `side` : 38
— `mitm` : 17

Ces résultats indiquent que la position `peer` est de loin la plus exploitée par l'algorithme, suggérant une vulnérabilité accrue de ces points d'accès dans le système. Les positions `side` et `mitm`, bien que moins fréquentes, apparaissent également comme vecteurs d'attaque potentiels.

Cette analyse démontre que XCS peut servir d'outil de diagnostic pour révéler les **points faibles topologiques** du système modélisé. Ces résultats pourraient ainsi orienter les efforts de sécurisation vers les interfaces les plus critiques.

7 Conclusion

7.1 Apports et limitations de XCS dans la simulation

Les résultats expérimentaux montrent que l'algorithme XCS apprend efficacement dans des environnements simples. Pour les modèles *VerySmall* et *Small*, les taux de réussite atteignent respectivement 96 % et 80 %, avec un temps d'exécution très réduit par épisode. Ces performances démontrent que XCS est bien adapté aux systèmes compacts, où l'espace d'état reste limité.

Cependant, à partir du modèle *Medium*, les performances se dégradent nettement : le taux de réussite chute à 40 % et le temps d'exécution augmente significativement. Sur le modèle *Large*, aucun apprentissage concluant n'a été observé, le taux de réussite tombant à 0 %. Cela souligne les limites de XCS en matière de scalabilité et d'efficacité dans des environnements complexes.

Deux facteurs expliquent ces limitations. D'une part, XCS explore des actions même lorsqu'elles ne sont pas disponibles, ce qui introduit un temps d'attente important pour identifier celles qui sont exécutables. Par exemple, si un système possède 395 transitions possibles mais que seul un sous-ensemble de 10 est disponible dans un état donné, la probabilité de sélectionner une action valable est très faible (10/395). D'autre part, la taille de l'espace d'état croît rapidement avec la complexité du système : pour un système composé de 78 modules, il faut encoder 78×2 bits, ce qui complexifie l'apprentissage.

7.2 Pistes d'amélioration futures

Plusieurs améliorations pourraient être envisagées pour améliorer l'efficacité de XCS :

— Réduire l'espace d'état à l'aide de techniques d'abstraction ou de regroupement des états similaires.

— Informer XCS des transitions valides à chaque état, en pénalisant directement les actions non exécutables, afin d'accélérer l'apprentissage.

— Coupler XCS à d'autres méthodes d'apprentissage, comme les réseaux de neurones ou le *deep reinforcement learning*, pour améliorer sa capacité de généralisation.

7.3 Conclusion finale

En résumé, XCS représente une approche prometteuse pour la simulation de cyberattaques dans des systèmes autonomes, mais ses performances restent limitées aux environnements de complexité faible à modérée. Dans des contextes plus complexes, son efficacité diminue fortement, avec des taux de réussite quasi nuls. Ces résultats montrent l'importance d'améliorer l'algorithme ou de l'intégrer à des approches hybrides pour qu'il soit applicable à des systèmes autonomes à grande échelle.

Glossaire

Apprentissage par renforcement : Méthode d'apprentissage automatique où un agent apprend par essais/erreurs, en recevant des récompenses ou des punitions selon ses actions.

Fitness : Mesure de la performance d'un individu (ou règle) dans un algorithme génétique : plus elle est élevée, plus l'individu est adapté.

Mutation : Modification aléatoire d'un individu dans un algorithme génétique, utilisée pour maintenir la diversité des solutions.

Fonction abstraite (abstract function) : Fonction définie dans une classe mère (souvent en programmation orientée objet) sans implémentation concrète. Elle doit être redéfinie (implémentée) dans une classe dérivée. En Python, on utilise souvent abc.ABC et @abstractmethod pour les déclarer.

Mapping : Association entre deux ensembles de valeurs. Ici, on utilise un mapping pour traduire des états (ex : FUNC) en bits (ex : [0, 0]).

Automate Structure représentant les modules du véhicule et leurs connexions. Chaque nœud est un module, chaque arête une dépendance ou communication.

Références

[1] Jaume Bacardit, Will Neil Browne, Jan Drugowitsch, and Ester Bernadó. Learning classifier systems. https://www.researchgate.net/publication/258407313_Learning_Classifier_Systems, 2013.

[2] Bibliothèques open source en python (xcs). https://github.com/hosford42/xcs/tree/master/xcs, 2023.

[3] CameBush. Découvrir le q-learning : Apprentissage par renforcement simplifié. https://www.youtube.com/watch?v=lacUlg3pO3w&ab_channel=CameBush, 2024.

[4] Myriam Emilion. Algorithme q-learning : apprentissage par renforcement. https://www.jedha.co/formation-ia/algorithme-q-learning, 2025.

[5] Catherine Escazut. Contribution aux systèmes de classifieurs : étude des mécanismes de spécialisation et extension du langage de représentation. https://theses.fr/1995NICE4843, 1995.

[6] Guillaume Hutzler, Hanna Klaudel, Witold Klaudel, Franck Pommereau, and Artur Rataj. An autonomous vehicle in a connected environment : case study of cyber-resilience, 2020.

[7] Data Science Team. Q-learning - apprentissage automatique - data science. https://datascience.eu/fr/apprentissage-automatique/q-learning/, February 2021.

[8] Stewart W. Wilson. State of xcs classifier system research. https://www.researchgate.net/publication/2817318_State_of_XCS_Classifier_System_Research, 2001.